Majlis

A moderated council of models—not a single optimising oracle. You set the agenda; the Curator shapes the panel; answers compete, clarify, and earn (or lose) trust.

THESIS

Reasoning improves when opinions can disagree

One model tends to hedge, over-explain, and hallucinate with high confidence. Real deliberation—assumptions surfaced, weak claims challenged—needs **multiple voices** and a **human moderator** who can steer, cut, and reward.

Majlis is that pattern productised: parallel responses, structured rounds, explicit kicks, and a small “Curator” model that routes, synthesises, and nudges—without replacing the user’s judgment.

PROBLEM

What’s broken in chat-with-one-LLM

- **No comparability.** You get one narrative. Hard to see trade-offs or alternative framings without hand-rolling your own adversarial prompts.
- **Length inflation.** Models optimise for completeness, not clarity—exactly wrong when you need a decision or a crisp critique.
- **Static trust.** Users can’t down-rank a bad answer in a way the system remembers; vendors rarely expose per-model accountability in the UI.

AUDIENCE

Who this is for

- **Builders & researchers** comparing approaches (architectures, methodologies, risks).
- **Power users** who want structured debate instead of endless follow-ups to one bot.
- **Hackathon / demo contexts** where showing multi-GPU inference and real product UX matters.

DESIGN GOAL

Concise turns, visible panelists, persistent history, and hooks for reputation so “good models” and “noise” separate over sessions—not just within one chat.

PRODUCT

How Majlis works (user journey)

- **Ask once.** You post a question. The Curator classifies the topic and recommends a shortlist from the available pool.
- **Pick the panel.** You choose who gets a seat and toggles capabilities (e.g. web search, “fast” mode) per model before the room opens.
- **Round-based discussion.** Each of your prompts starts a round: all active models answer in parallel (streamed), shown as stacked cards so you can scan without scrolling a wall of text.
- **Curator moves.** After a round, you trigger structured prompts—challenge, find consensus, devil’s advocate—so the next turn is *inter-model dialogue*, not a blind re-ask.
- **Kick + structured feedback.** Removing a panelist excludes their prior lines from *follow-up* discussion context so the room doesn’t keep “debating ghosts.”

CURATOR

Not a fifth participant—an operator

The Curator is a **small, fast model** used for: routing recommendations, optional warnings on weak reputations, **end-of-round synthesis** (agreement vs tension in one or two sentences), shallow-response checks after “go deeper,” and **server-defined discuss prompts** (`GET /curator/discuss-prompts`) so the UI isn’t hard-coding debate choreography.

This keeps the product feel intentional: the user sees “Curator move: Challenge” as a first-class action, not an opaque system message.

TECHNICAL CORE

Inference & orchestration

- **vLLM on AMD (ROCm)**—one OpenAI-compatible server per panel model (typ. 8B-class instruct checkpoints) plus a dedicated curator endpoint.
- **Parallel streaming.** `POST /discuss` fans out concurrent generation, merges Server-Sent Events, and persists each completed assistant message with layer metadata (`surface`, `depth`, `discuss`, `curator`).
- **Token discipline.** Discuss rounds use tighter caps so outputs stay comparable; depth rounds can expand when the user explicitly goes deeper on one panelist.

REQUEST SURFACE (CONCEPTUAL)

```
POST /discuss
  { room_id, message, target_participant_id?, mode?: "discuss",
    force_web_search?: bool }
→ SSE: { participant_id, model_id, chunk | done, layer }
```

DATA & STATE

Rooms, messages, reputation

- **SQLite + SQLModel** for rooms, participants, messages, and score records—simple to ship, easy to snapshot for demos.
- **History API** reloads the full transcript with `model_id` and display names so the UI colours and stacks stay consistent after refresh.
- **Dismissals** store multi-label feedback strings; backend adjusts reputation and can suggest replacements—closing the loop between UX and scoring.

TOOLS

Grounding without drowning in context

Tavily backs optional web retrieval: either per-panelist when “web search” was enabled at join, or a **single shared search** for the whole round when the user toggles “Web this round”—one query, results prepended for every active model so behaviour stays aligned.

The Curator’s synthesis and the discuss instructions both emphasise concision so search snippets augment answers instead of becoming copy-paste padding.

FRONTEND

Experience layer

- **React + Vite + TypeScript**, Tailwind v4, **shadcn/ui** for production-grade primitives (not bespoke CSS soup).
- **Clerk** for auth; tokens refreshed before each streamed call to avoid silent 401s on long sessions.
- **Cloudflare Pages** for static hosting at the edge; API on the GPU droplet behind Nginx and Cloudflare DNS/proxy.

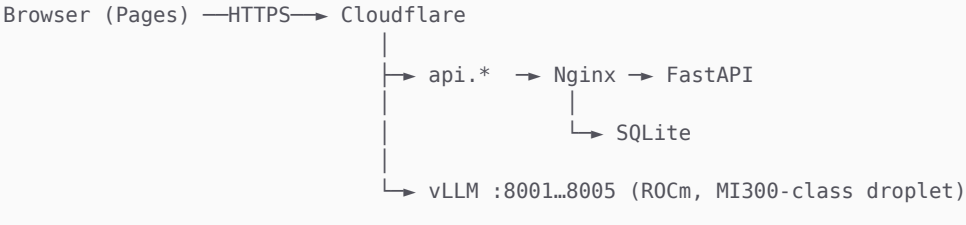
SECURITY & OPS

Trust boundaries we care about

- **JWT verification via JWKS** (Clerk)—no “trust this header” shortcuts on mutating routes.
- **CORS** locked to known frontend origins; secrets stay on the server (Tavily, HF, Clerk secret, model keys).
- **Deployment reality:** CI SSH to a home-lab GPU host often fails (firewall); production updates may be rsync + container restart—documented so the team isn’t blocked.

SYSTEM VIEW

End-to-end path



SSE streams flow from FastAPI through Nginx to the browser; message bodies are committed after each stream completes so history survives refresh and re-entry.

TRADE-OFFS

What we optimised for—and what we didn’t

- **For:** Demo clarity, multi-model parallelism, and a believable “product” narrative in hackathon time.
- **Against:** Cross-room analytics, enterprise SSO, fine-grained billing, and exhaustive prompt-injection hardening—those are next-layer work.
- **Compute:** Five 8B servers are heavy; the architecture assumes a capable GPU host, not a laptop-only plan.

ROADMAP (HONEST)

What would make this production-grad

- **Voting / resolution objects**—explicit “the room leans toward X” with logged rationale.
- **Richer tool schema**—code execution, structured citations, per-claim confidence if the stack allows.
- **Postgres + queues** if rooms need concurrency beyond SQLite’s comfort zone.
- **Evaluation harness**—automatic runs on fixed question sets to regress model mix and prompt quality.

DEMO SCRIPT

60 seconds that land

1. Open Majlis → show **marketing page** vs instant login wall.
2. Create a room → Curator recommendations → **select panel + capabilities**.
3. Fire a question → **parallel streams** → stacked cards.
4. Trigger a **Curator move** → watch inter-model discuss; toggle **web round** once.
5. **Kick** someone with multi-reason feedback → next round ignores them in context.

CLOSE

Majlis = deliberation as a product primitive

Multi-LLM is not “more text.” It’s **governance**: who speaks, who stays, what the room is asked to do next. We built the smallest stack that proves that thesis on real hardware.

majlis.mohessaid.com · api.mohessaid.com · repo + docker-compose for full repro.